
Análisis de Calidad del Software

Aplicación web de objetos perdidos y encontrados

No me la ratiai

Informe de evaluación técnica y académica
Basado en ISO/IEC 25010

Proyecto: No me la ratiai

Materia: Calidad de Software

Institución: Universidad Cooperativa de Colombia

Fecha: 25 de marzo de 2026

Tabla de Contenidos

1. Introducción
2. Descripción de la Aplicación
3. Metodología de Evaluación
4. Reporte Actual y Evidencia Técnica
5. Análisis con ISO/IEC 25010
 - 5.1. Resumen por característica
 - 5.2. Características con mayor evidencia
 - 5.3. Fallas del sistema según ISO/IEC 25010
6. Limitaciones del Prototipo
7. Oportunidades de Mejora
8. Conclusiones

Análisis de Calidad del Software

Proyecto: No me la ratiai

Tipo: Prototipo frontend con datos mock

Materia: Calidad de Software

Fecha del informe: 25 de marzo de 2026

Resumen ejecutivo: este informe analiza la calidad de la aplicación No me la ratiai como prototipo web académico orientado al reporte y recuperación de objetos perdidos. La evaluación se apoya en evidencia funcional, documentación QA, pruebas automatizadas y un mapeo explícito contra ISO/IEC 25010.

Estado verificado: 5/5 pruebas OK | lint: 0 errores, 7 warnings | build: exitoso

1. Introducción

Esta es una aplicación web diseñada para registrar objetos encontrados, facilitar reclamos y mostrar un perfil con incentivos para la devolución responsable. En esta versión académica, la app fue ajustada para una exposición centrada en calidad de software, con localización visible a Pasto, Colombia, simplificación visual y evidencia básica de aseguramiento de calidad.

El objetivo de este documento es presentar un análisis técnico y académico del estado actual de la aplicación, relacionando las características observables del sistema con la norma ISO/IEC 25010. El informe no pretende demostrar que la app sea un producto listo para producción, sino evidenciar cómo un prototipo frontend puede analizarse críticamente desde la perspectiva de calidad.

Al tratarse de un prototipo sin backend real, parte del análisis se apoya en evidencia directa del frontend, pruebas de interfaz, estructura del código y limitaciones explícitas del sistema.

2. Descripción de la Aplicación

La aplicación modela un flujo simple de objetos perdidos y encontrados. El usuario puede iniciar sesión, revisar publicaciones, abrir el detalle de un hallazgo, responder preguntas de verificación para reclamarlo y publicar nuevos objetos encontrados. Además, puede consultar un perfil con puntos, insignias y recompensas canjeables.

2.1. Flujos visibles en la app

- **Login:** acceso inicial mediante botón de ingreso.
- **Inicio:** consulta de hallazgos con búsqueda y filtros por categoría.
- **Publicar:** formulario para registrar un objeto encontrado.
- **Reclamar:** validación con preguntas de seguridad del hallador.
- **Perfil:** visualización de ubicación, estadísticas, insignias y recompensas.

2.2. Contexto y localización

Uno de los ajustes principales del proyecto fue la localización explícita a Pasto, Colombia. Esto se refleja en los datos mock visibles en la pantalla principal y en el perfil, incluyendo referencias como Plaza de Nariño, Universidad de Nariño, Terminal de Transportes de Pasto y recompensas asociadas a marcas reconocibles en Colombia como Rappi, Cruz Verde, Juan Valdez, Cine Colombia y Éxito.

Aspecto	Evidencia en la app
Ubicación del usuario	Pasto, Colombia
Referencias locales	Plaza de Nariño, Universidad de Nariño, Unicentro Pasto, Centro histórico
Recompensas	Rappi, Cruz Verde, Juan Valdez, Cine Colombia, Éxito
Estilo visual	Interfaz sobria, en español y sin emojis visibles

3. Metodología de Evaluación

La evaluación de calidad se construyó a partir de varias fuentes complementarias. Primero, se revisó la estructura funcional de la app para identificar los flujos implementados. Segundo, se usó la matriz QA del proyecto como evidencia manual. Tercero, se verificó el estado técnico actual mediante pruebas automatizadas, lint y compilación.

3.1. Fuentes de evidencia usadas

1. **Documentación del proyecto:** README con descripción, comandos y relación inicial con ISO/IEC 25010.
2. **Matriz QA:** casos de prueba manuales documentados para login, perfil, recompensas, publicar y reclamar.
3. **Pruebas automatizadas:** suite en Vitest y React Testing Library.
4. **Verificación estática:** ejecución de lint para detectar problemas de consistencia técnica.
5. **Verificación técnica:** build de producción para validar compilación.

3.2. Criterio de lectura del informe

El análisis se presenta con un enfoque balanceado y crítico. Cuando una característica de calidad tiene evidencia fuerte, se resalta. Cuando la evidencia es parcial por tratarse de un prototipo, se deja explícito como limitación. Esto evita sobrevalorar el sistema y mejora la validez académica del informe.

4. Reporte Actual y Evidencia Técnica

La app cuenta con una combinación de evidencia manual y automatizada. Para efectos del análisis académico, esa evidencia se reorganizó como una batería de verificación basada en ISO/IEC 25010, de modo que cada prueba aporte directamente a una característica de calidad del modelo.

4.1. Pruebas de evaluación ISO/IEC 25010 ejecutadas

Característica ISO 25010	Prueba o criterio aplicado	Evidencia actual	Resultado	Hallazgo o falla detectada
Adecuación funcional	Validar los flujos visibles principales: login, consulta de hallazgos, publicación, reclamo y perfil.	<code>src/test/index-smoke.test.tsx</code> , matriz QA y revisión funcional de pantallas.	Cumple	Los flujos existen, pero no hay persistencia real ni validación backend de los datos.
Usabilidad	Comprobar claridad visual, etiquetas, idioma, localización y consistencia del contenido mostrado al usuario.	<code>src/test/home-screen.test.tsx</code> , <code>src/test/profile-screen.test.tsx</code> , <code>src/test/publish-screen.test.tsx</code> .	Cumple	No existe evaluación formal de accesibilidad, contraste ni navegación completa por teclado.
Fiabilidad	Verificar estabilidad de navegación, disponibilidad de acciones principales y ausencia de regresiones visibles.	<code>npm test</code> , smoke test y compilación del proyecto.	Parcial	La app es estable como frontend, pero no hay tolerancia a fallos, recuperación de estado ni persistencia.
Seguridad	Revisar si el sistema protege acciones sensibles, identidad del usuario y datos del reclamo.	Inspección funcional del flujo de reclamo y login simulado.	Bajo	El login es simulado, no hay autenticación real, autorización, cifrado ni protección efectiva de datos.
Eficiencia de desempeño	Evaluar si la aplicación compila, carga y responde	<code>npm run build</code> y observación del comportamiento del frontend.	Parcial	No se ejecutaron métricas objetivas como Lighthouse, Web Vitals o

Característica ISO 25010	Prueba o criterio aplicado	Evidencia actual	Resultado	Hallazgo o falla detectada
	de forma liviana en su alcance actual.			profiling de rendimiento.
Compatibilidad	Comprobar adaptación básica a interfaz web responsive y ejecución en entorno estándar de navegador.	Revisión visual del prototipo y build exitosa.	Parcial	No existen pruebas cross-browser ni validación sistemática en distintos dispositivos.
Mantenibilidad	Revisar modularidad del código, separaciones internas y facilidad de modificación con apoyo de pruebas.	Estructura por componentes, datos mock, utilidades, tests y npm run lint .	Cumple	Persisten 7 warnings de lint que muestran oportunidades de mejora en componentes reutilizables.
Portabilidad	Comprobar si el proyecto puede instalarse, ejecutarse y compilarse con herramientas estándar.	README, Vite, Node.js, npm y build exitosa.	Cumple	No se documenta validación en múltiples sistemas operativos, aunque el stack favorece portabilidad.

4.2. Resultado técnico verificado

Comando	Propósito	Resultado
npm test	Validar comportamiento funcional y regresiones básicas del frontend.	5 archivos de prueba aprobados, 5 pruebas aprobadas.
npm run lint	Revisión estática del código.	0 errores, 7 advertencias de react - refresh/only-export-components .
npm run build	Confirmar compilación para producción.	Build exitosa.

Las advertencias de lint no bloquean la ejecución ni la compilación, pero sí indican oportunidades de mejora en la organización de algunos componentes reutilizables.

4.3. Evidencia manual documentada

La matriz QA registrada para el proyecto cubre diez casos: login, perfil localizado, recompensas colombianas, ausencia de referencias venezolanas, ausencia de emojis, flujo de publicar, flujo de reclamar, navegación al perfil, lint y build. En este informe, dicha matriz se interpreta como evidencia de apoyo para las características de adecuación funcional, usabilidad y fiabilidad definidas por ISO/IEC 25010.

5. Análisis con ISO/IEC 25010

ISO/IEC 25010 define un modelo de calidad con ocho características principales. En este proyecto, varias pueden demostrarse de forma directa a partir de la interfaz, la estructura del código y la evidencia QA; otras solo pueden evaluarse parcialmente debido a que la aplicación no implementa backend, base de datos ni autenticación real.

5.1. Resumen por característica

Característica	Valoración	Evidencia o comentario
Adecuación funcional	Alta	La app cubre los flujos esperados del caso académico: login, consulta, publicación, reclamo y perfil.
Eficiencia de desempeño	Media	La interfaz es ligera y compila correctamente, pero no hay métricas de rendimiento formales.
Compatibilidad	Media	Es una app web responsive para navegador; no se reportan pruebas cruzadas extensas de plataformas.
Usabilidad	Alta	Interfaz clara en español, navegación coherente, identidad local y reducción de ruido visual.
Fiabilidad	Media-Alta	Las pruebas automatizadas y el smoke test reducen regresiones en flujos clave.
Seguridad	Baja-Media	Existe verificación por preguntas en reclamo, pero no hay autenticación ni protección real de datos.
Mantenibilidad	Alta	Separación en componentes, datos mock, utilidades y suite de pruebas independiente.
Portabilidad	Alta	Puede ejecutarse con Node.js y npm mediante comandos estándar de Vite.

5.2. Características con mayor evidencia

5.2.1. Adecuación funcional

La adecuación funcional se evidencia en que los objetivos del prototipo están representados en flujos concretos: iniciar sesión, revisar objetos encontrados, registrar nuevos hallazgos, iniciar un reclamo y consultar un perfil con incentivos. Para una materia de calidad, esto demuestra que el producto implementa funciones alineadas con su propósito académico.

Adicionalmente, la localización a Pasto no es solo estética; también fortalece la pertinencia del contenido porque hace que el prototipo se sienta contextualizado al entorno del usuario final

planteado.

5.2.2. Usabilidad

La usabilidad es una de las fortalezas principales del proyecto. La app trabaja completamente en español, usa etiquetas visibles, botones claros y una navegación inferior consistente. La eliminación de emojis visibles reduce ruido visual y hace que la experiencia se perciba más sobria y académicamente más cuidada.

El rediseño del perfil, las insignias y las recompensas con marcas reconocibles mejora la comprensión visual del sistema y la coherencia del contexto colombiano. Aunque no hay una evaluación formal de accesibilidad, sí existe un esfuerzo claro por jerarquía visual y consistencia.

5.2.3. Fiabilidad

En un prototipo frontend, la fiabilidad se observa sobre todo en la estabilidad de los flujos más sensibles. Aquí esa evidencia existe por medio de una suite automatizada que valida inicio, perfil, publicación, reclamo y navegación básica. También existe una matriz QA manual que amplía el control funcional.

La app compila correctamente y las pruebas ejecutadas hoy pasaron por completo. Esto no garantiza confiabilidad productiva a gran escala, pero sí demuestra control sobre regresiones en el alcance actual del proyecto.

5.2.4. Mantenibilidad

La mantenibilidad está bien representada por la organización del código. La aplicación separa pantallas, navegación, datos mock, componentes auxiliares y pruebas. Esta distribución facilita entender el comportamiento del sistema sin necesidad de recorrer un único archivo monolítico.

Además, el uso de pruebas específicas por pantalla ayuda a sostener cambios futuros. Las advertencias de lint muestran que todavía hay espacio para mejorar ciertos archivos reutilizables, pero no invalidan la base estructural actual.

5.2.5. Características con evidencia parcial

Eficiencia de desempeño: la app es liviana y el build es exitoso, pero no existen mediciones como Lighthouse, Web Vitals o profiling real.

Compatibilidad: al ser una web app construida con Vite y React, es razonable afirmar compatibilidad básica con navegadores modernos, aunque no hay pruebas cruzadas documentadas.

Seguridad: el flujo de reclamo usa preguntas de validación, pero esto no reemplaza autenticación, autorización, cifrado ni protección real de datos personales.

Portabilidad: el proyecto es portable en entornos comunes de desarrollo porque solo requiere Node.js, npm y un navegador.

5.3. Fallas del sistema según ISO/IEC 25010

ID	Falla detectada	Característica afectada	Impacto en calidad
F-01	Login simulado sin autenticación real.	Seguridad	El sistema no garantiza identidad del usuario ni control de acceso sobre acciones sensibles.
F-02	Reclamos y publicaciones sin persistencia real de datos.	Fiabilidad / Adecuación funcional	El flujo funciona en interfaz, pero no asegura continuidad, trazabilidad ni consistencia de información.
F-03	Ausencia de métricas objetivas de rendimiento.	Eficiencia de desempeño	No se puede demostrar con evidencia cuantitativa el tiempo de carga, respuesta o consumo de recursos.
F-04	Sin validación formal de accesibilidad ni de navegación asistida.	Usabilidad	La interfaz es clara visualmente, pero no hay garantía de uso adecuado para todos los perfiles de usuario.
F-05	No hay pruebas cruzadas entre navegadores y dispositivos.	Compatibilidad	La app puede comportarse de forma distinta fuera del entorno donde fue demostrada.
F-06	Existen advertencias de lint en componentes reutilizables.	Mantenibilidad	Aunque no bloquean la ejecución, señalan áreas del código que conviene fortalecer para cambios futuros.

En términos académicos, estas fallas no invalidan el proyecto, pero sí delimitan con claridad qué características de ISO/IEC 25010 están realmente soportadas y cuáles solo tienen evidencia parcial.

6. Limitaciones del Prototipo

El valor académico del proyecto depende de reconocer con claridad sus límites. No debe presentarse como un sistema productivo listo para despliegue institucional, sino como un prototipo frontend enfocado en experiencia de usuario, localización y evidencia QA básica.

- No existe backend real ni persistencia de datos.
- El login es simulado y no maneja autenticación real.
- Las preguntas de seguridad del reclamo son demostrativas, no un mecanismo robusto de protección.
- No hay mediciones formales de rendimiento, accesibilidad o compatibilidad cross-browser.
- Las recompensas, hallazgos y estados del sistema provienen de datos mock.

Reconocer estas limitaciones fortalece el informe, porque demuestra criterio de ingeniería y evita afirmar capacidades que el sistema no implementa.

7. Oportunidades de Mejora

A partir del estado actual, se pueden proponer mejoras concretas que aumentarían la calidad del sistema y ampliarían la cobertura real de ISO/IEC 25010.

1. Incorporar backend y persistencia para gestionar hallazgos, reclamos y perfiles reales.
2. Implementar autenticación y autorización para proteger acciones sensibles.
3. Agregar pruebas end-to-end reales con Playwright o Cypress sobre el flujo completo.
4. Medir rendimiento con herramientas objetivas como Lighthouse o Web Vitals.
5. Realizar una revisión formal de accesibilidad y navegación asistida por teclado.
6. Reducir warnings de lint en componentes reutilizables para fortalecer mantenibilidad.

8. Conclusiones

Esta app presenta una base sólida como ejercicio académico de calidad de software. La app cumple de manera convincente con adecuación funcional, usabilidad, fiabilidad básica y mantenibilidad, que son precisamente las características de ISO/IEC 25010 con mejor evidencia en el contexto actual.

Al mismo tiempo, el proyecto muestra madurez al documentar pruebas, disponer de una matriz QA y aceptar que seguridad, rendimiento y compatibilidad todavía solo tienen evidencia parcial. Esta combinación de fortalezas verificables y límites explícitos hace que el informe sea defendible en un curso de calidad de software.

En síntesis, el proyecto no demuestra un producto terminado para producción, pero sí evidencia una aplicación académica bien estructurada, verificable y suficientemente analizable desde el marco de ISO/IEC 25010.